

Xeno Data Analyst Assignment

Report

1.My Approach:

The first useful step was to read the assignment and the supplied README carefully. The README already defines the business rules, so there was no reason to spend time guessing what `parent_id` means or inventing a new definition for a retry. Instead, I treated those rules as the specification and used the data to determine where they actually affected the October 2026 population.

That distinction mattered. The task was not simply 'write SQL that returns 22'. The requested reconciliation bridge had to show the starting point, the mismatch, what I checked next, and why each adjustment was justified. My approach was therefore to start with the most obvious count and progressively make the logic closer to the Finance definition.

2. What the README told us upfront

Before running queries, I wrote down the rules that were already explicit in the provided data dictionary. This gave me a working mental model and, importantly, prevented me from treating known business definitions as discoveries.

Given by the README	How it shaped the investigation
campaign	One row per campaign. <code>parent_id</code> links a retry to the earlier campaign.
communication_log	One row per individual send attempt.
Retry chain	A chain such as A -> B -> C is one underlying communication. A customer appearing several times inside that chain counts once.
Standalone campaign	A campaign with no retry chain is different: repeated sends for the same customer are legitimate separate events.
Reporting eligibility	A campaign counts only when its creation workflow has cleared and <code>processing_status</code> is processed. <code>approval_awaiting</code> does not count even if log rows exist.
Scope	<code>merchant_id = 501</code> , October 2026, <code>communication_type = '2'</code> .

So the central question became: how do we translate those written rules into a reconciliation that can be checked against the actual rows?

3. The mental model: what could make a naive count too high?

Before changing the SQL, I classified the plausible reasons the raw log could exceed Finance's number. There were three obvious buckets:

A. Rows that should not be reportable

A send attempt can exist in communication_log even when the campaign itself is not yet cleared for official reporting.

B. Multiple attempts that are actually retries

A retry can generate another communication_log row for the same customer, but the business metric counts that customer once within the retry family.

C. Repeated sends that are not retries

A repeated customer in a standalone campaign can still represent two genuine send events. Those rows should not be removed simply because the customer ID repeats.

This was the main reasoning checkpoint. It ruled out two tempting but incorrect shortcuts: counting every row forever, and globally doing COUNT(DISTINCT customer_id). The data definition requires a conditional counting rule based on campaign lineage.

4. Investigation 1 — establish the naive baseline

I started with the simplest possible view of the data: treat each communication_log row as an independent event and count the rows for the requested scope.

```
SELECT COUNT(*) AS naive_count
FROM communication_log
WHERE merchant_id = 501
      AND communication_type = '2'
      AND sent_time >= '2026-10-01'
      AND sent_time < '2026-11-01';
```

Result: 30

The gap was therefore 8 rows versus Finance's target of 22. At this stage, I did not assume those eight rows were duplicates. The next question was which business rule could explain the first part of the difference.

5. Investigation 2 — check whether all logged sends are reportable

The README explicitly warns that the send pipeline can run ahead of campaign approval. That made campaign status the most direct next check. I joined the log to campaign and applied the reporting eligibility rule.

```
SELECT COUNT(*) AS eligible_sends
FROM communication_log cl
JOIN campaign c
  ON cl.communication_id = c.id
WHERE cl.merchant_id = 501
      AND cl.communication_type = '2'
      AND cl.sent_time >= '2026-10-01'
      AND cl.sent_time < '2026-11-01'
      AND c.creation_status = 'approved'
      AND c.processing_status = 'processed';
```

Result: 26

This removed exactly four rows. Looking at the campaign-level records showed that those four rows belonged to campaign 9004, whose creation_status is approval_awaiting. The rows exist, but under the provided reporting definition they are not part of the official population.

6. Reconciliation Checkpoints:

What changed?	Why?	Running total
Naive communication-log rows	Every matching log row initially counted.	30
Remove campaign 9004	approval_awaiting is not reportable.	26

7. Investigation 3 — look at campaign lineage

With eligibility accounted for, the remaining six-row gap had to come from how sends are counted inside the campaign structure. At this point the parent_id rule in the README became the useful lens: rather than treating campaign IDs as unrelated buckets, I grouped campaigns that belong to the same retry lineage.

The relevant campaign structure is:

```
9001
| -- 9002
|   `-- 9003
|     `-- 9004 (not reportable)
```

9101 (standalone)

```
9201
  `-- 9202
```

8. Retry family 9001 -> 9002 -> 9003

The first retry chain contains 13 communication-log attempts in total, but the customers covered by the chain are C1 through C10. The most illustrative cases are C2 and C3: C2 appears as a failed attempt in 9001 and a successful attempt in 9002, while C3 appears across all three campaigns before reaching a successful attempt. These are exactly the patterns the README describes as retries rather than three independent communications.

Therefore the chain contributes 10 customers, not 13 attempts. This is an adjustment of -3, taking the running total from 26 to 23.

9. Retry family 9201 -> 9202

The second retry chain follows the same rule. It contains 6 attempts but only 5 distinct customers because D1 appears in the original campaign and its retry. Collapsing that retry family removes one over-counted attempt.

Running total: 23 -> 22.

10. The check that prevented an incorrect global DISTINCT

At this point it would have been tempting to simplify everything to COUNT(DISTINCT customer_id). I explicitly checked the standalone campaign because the README says that a customer may appear more than once against the same communication_id and that this is a separate event from a retry.

Campaign 9101 is the important example. It contains 7 send rows but only 6 distinct customers because C20 appears twice on different dates. Because 9101 is standalone, those two rows are retained as two legitimate send events.

This was a useful sanity check even though it did not change the final 22. It showed that the metric cannot be implemented as a single global DISTINCT or a simple row count.

11. Final reconciliation

Stage	Adjustment	Reason	Total
Naive count	—	Raw send attempts in scope	30
Eligibility	-4	Exclude non-reportable campaign 9004	26
Retry family 9001/9002/9003	-3	13 attempts -> 10 distinct customers	23
Retry family 9201/9202	-1	6 attempts -> 5 distinct customers	22

Finance target_base reproduced: 22

12. What surprised me

The most useful surprise was that a repeated customer ID does not automatically mean a duplicate. C20 appears twice in standalone campaign 9101, but the two sends are legitimate events. On the other hand, repeated customers across a retry chain must be collapsed. That contrast is easy to miss if the problem is approached as a generic duplicate-removal task.

A second interesting detail was campaign 9004: communication-log rows existed even though the campaign was still approval_awaiting. That made the eligibility join an important part of the reconciliation rather than an optional cleanup step.